



Walchand College of Engineering, Sangli

(Government Aided Autonomous Institute)

Department of Computer Science and Engineering

Project Synopsis

MABSplit — Accelerated Random Forest & Decision Tree Training via Multi-Armed Bandit Node Splitting

Bachelor of Technology
in
Computer Science and Engineering

Submitted by

Kedar Somnath Dhotre	23510090
Aditya Rajkumar Pansare	23510080
Onkar Gunaji Rane	23510086
Dhanashree Maid	23510088

Under the Guidance of

P.B.Kulkarni
Assistant Professor

2026-2027

Contents

1. Introduction	2
2. Customer Connect Details	4
3. Literature Review	5
4. Objectives	7
5. Sustainable Development Goal (SDG) Mapping	8
6. Scope of the Project	9
7. Proposed Methodology	10
8. Expected Outcomes	18
9. Feasibility Study	19
References	20

1. Introduction

1.1 Background

Standard Random Forests and Gradient Boosted Decision Trees (GBDTs) remain the dominant algorithms for tabular data across finance, medical diagnostics, and fraud detection, routinely outperforming deep learning baselines on structured data benchmarks. However, they incur prohibitive $O(N \cdot F \cdot B)$ computational complexity at every tree node by exhaustively scanning all N data samples across F features and B split thresholds – for a dataset such as Higgs ($N \approx 11,000,000$, $F = 28$) this means roughly 3×10^8 point evaluations *per node*, repeated across every node of every tree in the forest. The MABSplit algorithm (Tiwari et al., NeurIPS 2022) reformulates this brute-force node splitting as a Multi-Armed Bandit (MAB) problem, treating each (feature, threshold) pair as an “arm” whose true impurity-reduction value is unknown and must be estimated from samples. By leveraging Upper Confidence Bounds (UCB) and variance propagation over sample histograms, MABSplit reduces the per-node sample complexity from linear $O(N)$ to logarithmic $O(K \log N)$, where K is the number of candidate arms. Conceptually, instead of reading every data point completely (like a judge reading 100 full essays), MABSplit acts as a smart scout – it reads small minibatches, tracks confidence intervals, and quickly discards obviously suboptimal splits, focusing computation only on competitive candidates. The algorithm carries a probabilistic correctness guarantee: with probability at least $1 - \delta$ it recovers the *exact* optimal split that brute-force scanning would have found, not an approximation.

1.2 Motivation

While deep neural networks scale efficiently via mini-batch SGD and GPU parallelization, exact decision tree construction lacks similar adaptive scaling, requiring expensive multi-socket CPU clusters for large-scale training. Tabular datasets dominate industry applications – an estimated majority of enterprise machine learning workloads (fraud scoring, credit risk, churn prediction, ad ranking) still rely on tree ensembles rather than deep networks – making the node-splitting training bottleneck a critical practical problem rather than a purely academic one. Accelerating the node-splitting subroutine by up to 100x without loss of classification/regression accuracy enables faster model iteration, lower cloud compute costs, and real-time tree learning on edge/resource-constrained devices where full-scan retraining is infeasible. Although MABSplit theoretically provides up to a 99% reduction in sample evaluations, the reference Python/NumPy implementation suffers from se-

vere constant-factor overheads (interpreter dispatch, repeated array allocation, unvectorized histogram updates) that hide these asymptotic gains in wall-clock terms – the original paper itself reports large sample-count reductions but comparatively modest wall-clock speedups on several benchmarks. This gap between theoretical sub-linear sample complexity and real-world wall-clock speedup is the central motivation for this project.

1.3 Problem Statement

The computational bottleneck of brute-force node splitting in decision trees and random forests hinders scalability on high-dimensional and large-scale tabular datasets. Exact decision tree construction requires sorting and scanning full feature arrays at every node split, scaling as $O(D \cdot N \log N \cdot F)$ for tree depth D . Although MABSplit theoretically reduces per-split sample complexity to $O(K \log N)$, the official Python implementation suffers from three concrete, code-verifiable inefficiencies: (1) fixed per-arm bookkeeping and confidence-bound computation impose execution overhead that exceeds brute-force scanning on small leaf nodes where $N < 1000$, a regime that constitutes the majority of nodes near the bottom of any moderately deep tree; (2) a fixed minibatch size of 100 samples per round causes wasteful over-sampling once the active arm set has shrunk to only 2–3 competitive candidates, since the same batch size is drawn regardless of how many arms remain; and (3) pure Python interpreter dispatch combined with repeated `np.concatenate`-style array reallocation inside the sampling loop prevents the theoretical $O(K \log N)$ sample reduction from translating into a proportional wall-clock speedup. This project targets all three inefficiencies directly.

2. Customer Connect Details

2.1 Customer Name

Open Source Community – ThrunGroup/FastForest Repository (<https://github.com/ThrunGroup/FastForest>), the reference implementation of the MABSplit algorithm (Tiwari et al., NeurIPS 2022)

2.2 Contact Person

Repository maintainers, ThrunGroup (Stanford University) – contacted via the repository’s public issue tracker rather than a named individual, since no direct maintainer contact is published on the repository.

2.3 Email / Phone

Public engagement channel: GitHub Issues on <https://github.com/ThrunGroup/FastForest/issues>.

2.4 Nature of Engagement

This is an open-source extension project rather than an industry-sponsored one: there is no contracted external customer. The “customer” is the FastForest open-source codebase and its maintainer community, engaged asynchronously – the team performs static analysis and benchmarking against the public repository and the NeurIPS 2022 paper, cross-checking derived formulas (UCB/LCB bounds, Gini/MSE variance propagation) against the published proofs, and may open GitHub issues/pull requests if project outcomes are proposed back upstream. Periodic review meetings with the internal project guide will be used to validate algorithmic modifications, review benchmark methodology, and approve experimental result interpretation before each phase gate in the work plan (Section 7.6).

3. Literature Review

1. **Mo Tiwari et al., 2022** – Proposed MABSplit algorithm which formulates split selection as a Multi-Armed Bandit problem. Achieves sublinear sample complexity $O(K \log N)$ to find the exact optimal split with probability $\geq 1 - \delta$, using successive elimination over UCB/LCB confidence intervals built from incrementally sampled minibatches. Limitation: Assumes i.i.d. mini-batch sampling; fixed histogram binning introduces spatial discretization error; confidence multiplier γ requires manual tuning for skewed label distributions; batch size and bin count are treated as static hyperparameters rather than adapted at runtime.
2. **Ke et al., 2017** – Developed LightGBM using Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB) to accelerate GBDTs. GOSS samples data instances based on gradient magnitude, retaining large-gradient instances and randomly sampling small-gradient ones, but is non-bandit and uses a single static sampling ratio. Limitation: Does not utilize confidence bounds for adaptive arm elimination, potentially discarding informative gradients prematurely, and provides no statistical guarantee of recovering the exact optimal split.
3. **Geurts et al., 2006** – Proposed Extremely Randomized Trees (Extra-Trees) which choose split thresholds randomly (rather than exhaustively searching for the best one) to reduce computational load, only optimizing the choice of feature. Limitation: Lacks any bandit-style adaptive sampling to guarantee near-optimal splits with minimal samples, relying entirely on randomization, which trades accuracy variance for speed and can miss critical split points on data with sharp decision boundaries.
4. **Jamieson et al., 2014** – Proposed lil' UCB, a near-optimal algorithm for best-arm identification in general fixed-confidence multi-armed bandit settings using upper confidence bounds with logarithmic sample complexity. Limitation: Designed for general-purpose best-arm identification with scalar rewards; does not account for the specific variance propagation structures (Gini impurity / MSE reduction) inherent in decision tree splitting, which MABSplit must derive separately for classification and regression.

3.1 Research Gap

Existing MABSplit implementation suffers from multiple unoptimized components identified via static code analysis of the public FastForest repository. *Author-stated gaps*: fixed uniform histogram binning misses optimal continuous split points, introducing a discretization error bounded by the bin width. *Code-observed gaps*: redundant feature sampling (updating bin edges even for arms already eliminated from the active set), frequent memory allocation (array concatenation performed inside the innermost sampling loop rather than pre-allocated), and pure Python interpreter bottlenecks that dominate wall-clock time once the sample count per round is small. *Algorithmic gaps*: bandit overhead exceeds brute-force cost on small leaf nodes ($N < 1000$), and the fixed batch size of 100 wastes compute in late rounds when only 2–3 arms remain active, since bookkeeping and bound computation are done per-arm regardless of active-set size. This project fills these gaps by introducing a Hybrid Depth Switch, Coarse-to-Fine Arm Pruning, Adaptive Minibatch Allocation, and a Vectorized Numba Engine to convert the algorithm’s theoretical sample-efficiency into measured wall-clock speedups.

4. Objectives

1. To implement a Node-Depth Adaptive Hybrid Solver (Hybrid-MABSplit) that dynamically switches between MABSplit and exact vectorized sorting based on a sample threshold N_{thresh} (tuned empirically per dataset, expected in the range 500–2000), eliminating MAB bookkeeping overhead on small leaf nodes where brute-force is already cheap.
2. To develop a Hierarchical Coarse-to-Fine Arm Pruning mechanism (H-MABSplit) that evaluates features at coarse histogram resolution ($B_{coarse} = 16$) to eliminate clearly non-competitive feature dimensions before expanding only the surviving features to fine resolution ($B_{fine} = 256$), reducing initial arm-evaluation memory and per-round bound computation on high-dimensional datasets.
3. To design an Adaptive Dynamic Minibatch Allocation (ADMA-MABSplit) strategy where minibatch size scales dynamically as $M_t = \max(M_{min}, \lfloor M_0 \cdot (|Active_t|/K)^\alpha \rfloor)$ with tunable decay exponent α , reducing wasteful sampling in late iterations when few arms remain contested.
4. To re-engineer the inner sampling and histogram accumulation loops into a Vectorized Numba JIT-compiled Memory-Reused MAB Engine (Fast-MAB) using pre-allocated NumPy buffers and @njit-compiled kernels, converting theoretical sample reductions into real-world wall-clock speedups.
5. To validate the proposed improvements rigorously on standard tabular datasets (Higgs, Adult, Covertypes) against the baseline MABSplit and scikit-learn, measuring sample query counts, wall-clock time, peak memory usage, and held-out model accuracy (F1 for classification, RMSE for regression) across at least 5 random seeds per configuration for statistical significance.

5. Sustainable Development Goal (SDG) Mapping

This project is mapped to the following Sustainable Development Goal, in consultation with the project guide: **Mapped SDG: 9 – Industry, Innovation, and Infrastructure** **Justification:**

Accelerating the training of Random Forests promotes efficient computational infrastructure and fosters innovation in core machine learning algorithms. By significantly reducing CPU cycles and compute costs – a 5x–10x wall-clock speedup translates directly to proportional reductions in energy consumption and cloud compute expenditure for tabular-data workloads – it enables sustainable scaling of AI infrastructure, allowing industries to deploy high-accuracy tabular models on resource-constrained edge devices without relying on energy-intensive GPU clusters. **Reference – List of Sustainable Development Goals**

(SDG):

Goal	Title	Goal	Title	Goal	Title	Goal	Title
1	No Poverty	5	Gender Equality	9	Industry, Innovation, and Infrastructure	13	Climate Action
2	Zero Hunger	6	Clean Water and Sanitation	10	Reduced Inequality	14	Life Below Water
3	Good Health and Well-being	7	Affordable and Clean Energy	11	Sustainable Cities and Communities	15	Life on Land
4	Quality Education	8	Decent Work and Economic Growth	12	Responsible Consumption and Production	16	Peace, Justice, and Strong Institutions
						17	Partnerships to achieve the Goal

Table 1: Sustainable Development Goals (SDG)

6. Scope of the Project

Included in Scope:

- Static codebase analysis and paper-to-code mapping of the official ThrunGroup/FastForest repository, documenting every deviation between the published algorithm and its reference implementation.
- Mathematical verification of UCB/LCB bounds and variance propagation formulas for both Gini impurity (classification) and MSE reduction (regression) objectives.
- Implementation of Hybrid-MABSsplit, H-MABSsplit, ADMA-MABSsplit, and Fast-MAB modifications as modular, independently-toggleable components so each can be ablated in isolation during evaluation.
- Experimental evaluation on standard tabular datasets (Higgs, Adult, Covertype, and a controlled synthetic dataset for boundary-case stress testing) across both classification and regression tasks.
- Comparative benchmarking against baseline MABSsplit and scikit-learn's exact solver, reporting sample-query counts, wall-clock time, memory footprint, and accuracy with confidence intervals across multiple seeds.

Excluded from Scope:

- Extension to Gradient Boosted Decision Trees (GBDT-MABSsplit) requiring derivation of Hessian-weighted variance estimation, left as future work.
- Distributed computing infrastructure across multiple cluster nodes; the project targets single-workstation, multi-core CPU execution only.
- Deployment and inference-time optimization on edge devices; the scope is limited to accelerating the training-time node-splitting subroutine.

7. Proposed Methodology

7.1 System Architecture / Approach

The proposed system modifies the core MABSplit node splitting subroutine via the following algorithmic workflow:

1. **Hybrid Depth Dispatch:** At node creation, if sample count $N < N_{thresh}$, dispatch to exact vectorized sorting (guaranteed optimal, cheap at small N). If $N \geq N_{thresh}$, initialize the MAB solver.
2. **Coarse-to-Fine Initialization:** Build feature histograms at $B_{coarse} = 16$ bins. Execute a preliminary short MAB sampling round to prune unpromising features whose coarse UCB falls below the best coarse LCB. Expand only surviving features to $B_{fine} = 256$ bins for fine-grained split search.
3. **Adaptive Minibatch Sampling:** Draw a batch of size $M_t = \max(M_{min}, \lfloor M_0 \cdot (|Active_t|/K)^\alpha \rfloor)$ from the current node’s data without replacement, where $|Active_t|$ is the number of arms still in contention at round t .
4. **Impurity & Variance Update:** Insert sampled values into the per-arm histograms. Compute the empirical impurity reduction $\hat{\Delta}_a$ and variance estimate σ_a^2 for all active arms using Welford-style online updates to avoid re-scanning accumulated samples.
5. **UCB/LCB Calculation:** Compute confidence bounds: $LCB_a = \hat{\Delta}_a - \gamma\sqrt{\sigma_a^2/n_a}$ and $UCB_a = \hat{\Delta}_a + \gamma\sqrt{\sigma_a^2/n_a}$, where n_a is the cumulative sample count for arm a and γ is the confidence multiplier derived from the target error probability δ .
6. **Arm Pruning:** Find UCB_{min} , the smallest upper bound across all active arms. Retain arms where $LCB_a \leq UCB_{min}$ (i.e. arms that could still plausibly be optimal). Terminate when $|Active| \leq 1$ or the full dataset has been sampled, at which point the surviving arm is returned as the split.

All sampling and histogram-accumulation loops will be re-written using Numba @njit JIT compilation and pre-allocated, memory-reused arrays to eliminate Python interpreter dispatch overhead and per-round allocation cost, which static profiling of the baseline identifies as the dominant contributor to wall-clock time at small-to-medium sample counts.

7.2 Tools and Technologies

- **Programming Language(s):** Python 3
- **Framework(s)/Library(ies):** NumPy, SciPy, Numba (for JIT compilation of hot inner loops), scikit-learn (baseline exact-solver comparison), Matplotlib/Seaborn (result visualization), pytest (unit and regression testing of split correctness)
- **Database:** In-memory NumPy arrays loaded from standard UCI/OpenML tabular datasets (Higgs, Adult, Covertypes) plus a generated synthetic dataset for controlled boundary-case testing
- **Hardware (if any):** Standard multi-core CPU workstation (no GPU required); benchmarking will fix thread count to isolate algorithmic gains from parallelism effects
- **Version Control / Collaboration:** Git/GitHub, forked from ThrunGroup/FastForest with all modifications tracked as reviewable commits/branches

7.3 Datasets

Dataset	Samples (N)	Features (F)	Task	Role
Higgs	$\sim 11,000,000$	28	Classification	Large-scale stress test
Adult (Census Income)	48,842	14	Classification	Medium-scale, mixed features
Covertypes	581,012	54	Classification	High-dimensional, multi-class
Synthetic (generated)	Configurable	Configurable	Both	Controlled boundary-case testing

Table 2: Benchmark datasets used for evaluation

7.4 UML Diagrams

- **Use-Case Diagram:** Depicts the researcher/user invoking forest training, configuring hyperparameters (N_{thresh} , B_{coarse} , B_{fine} , M_0 , α), and retrieving benchmark reports.

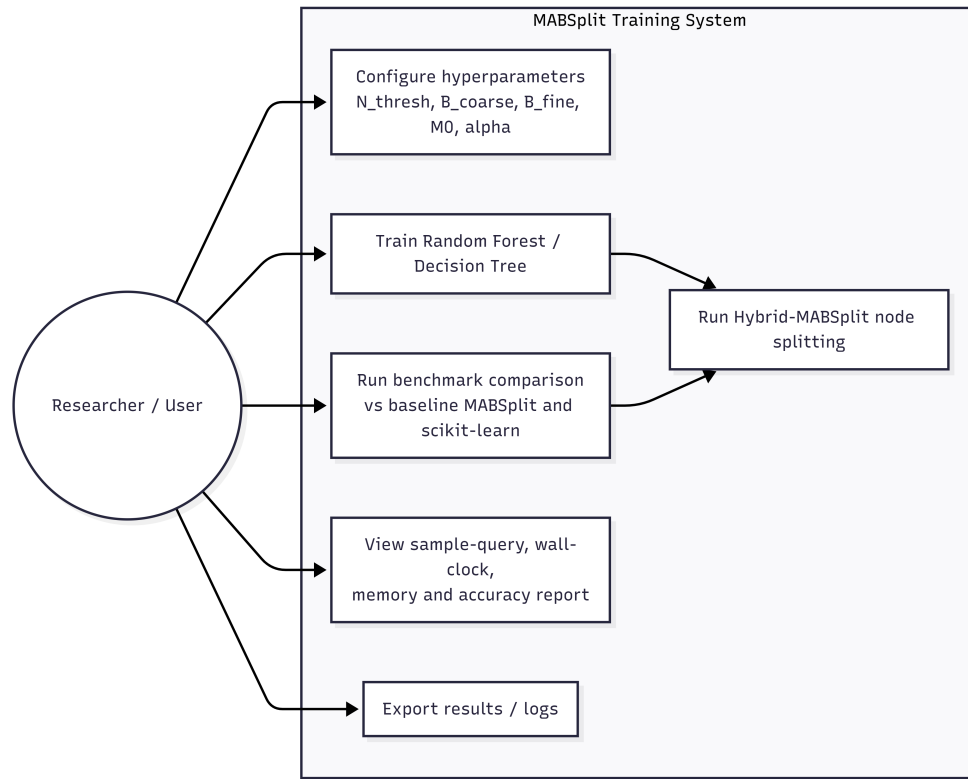


Figure 1: Use-Case Diagram of the Proposed System

- **Class Diagram:** Models the HybridSolver, ArmPruner, MinibatchAllocator, and FastMABEngine classes, their attributes (active-arm sets, histograms, confidence bounds), and their composition relationships within the overall tree-builder.

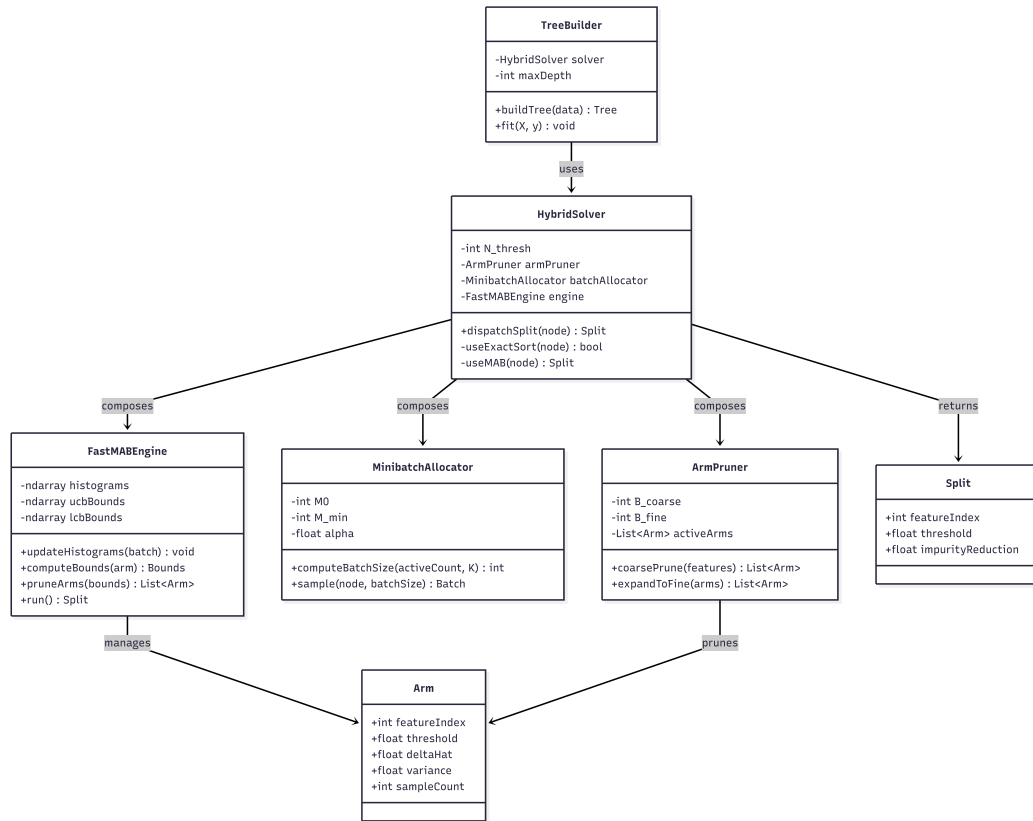


Figure 2: Class Diagram of the Proposed System

- **Sequence Diagram:** Traces a single node-split call from HybridSolver dispatch through coarse pruning, adaptive sampling rounds, and final arm selection, showing interaction with the Numba-compiled engine.

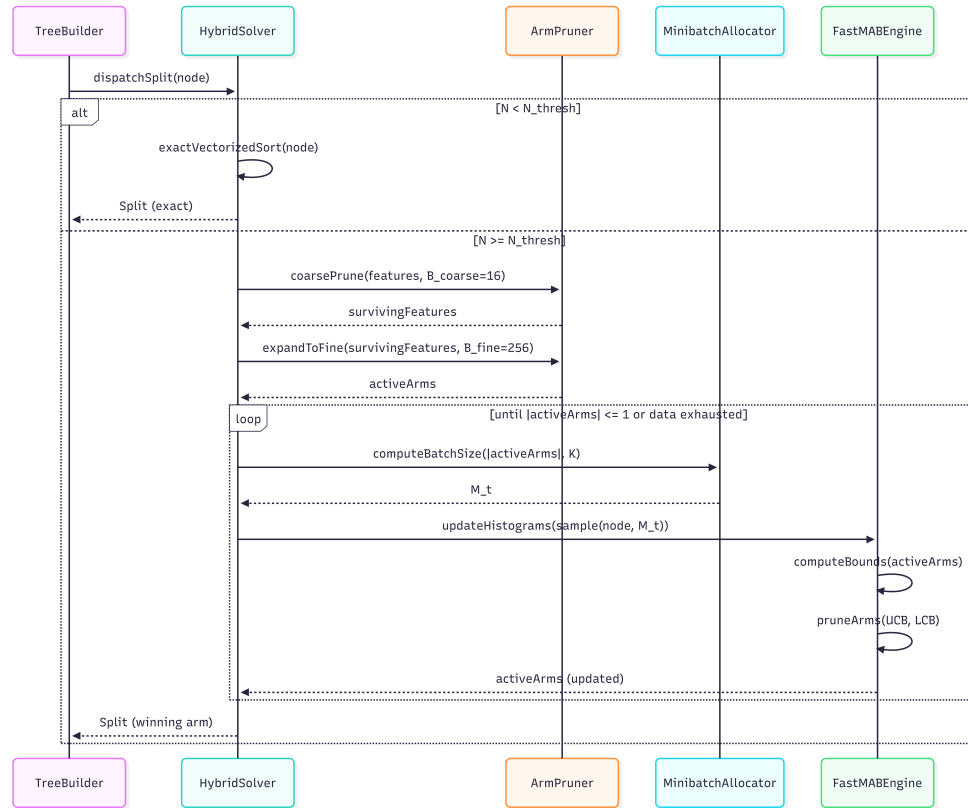


Figure 3: Sequence Diagram for Node-Split Execution

- **Activity Diagram:** Details the iterative sample-update-prune loop of the MAB solver, including the termination conditions ($|Active| \leq 1$ or full data exhausted).

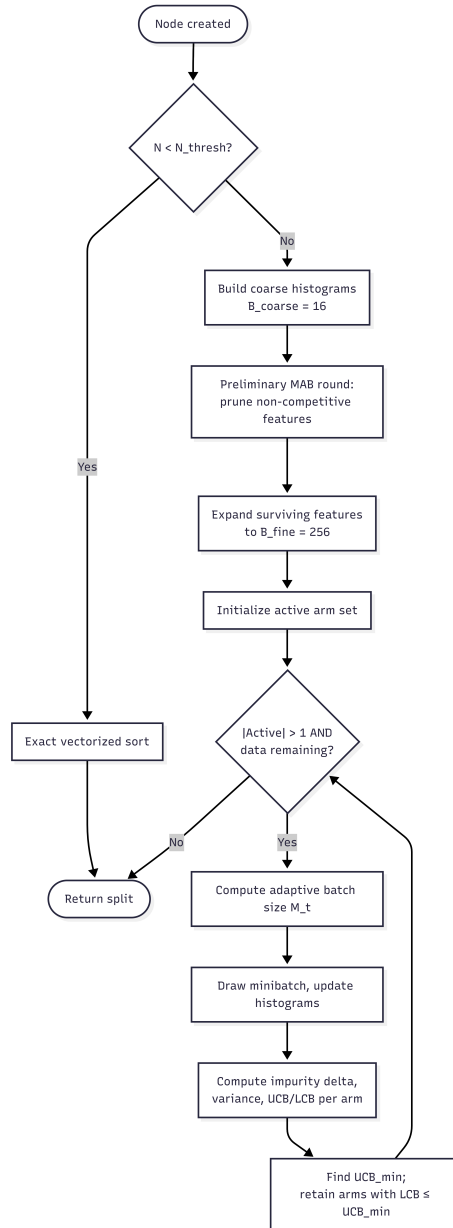


Figure 4: Activity Diagram of the MAB Solver

- **Deployment Diagram:** Shows the single-workstation CPU execution environment, with the Python driver process invoking JIT-compiled native code for the hot inner loops.

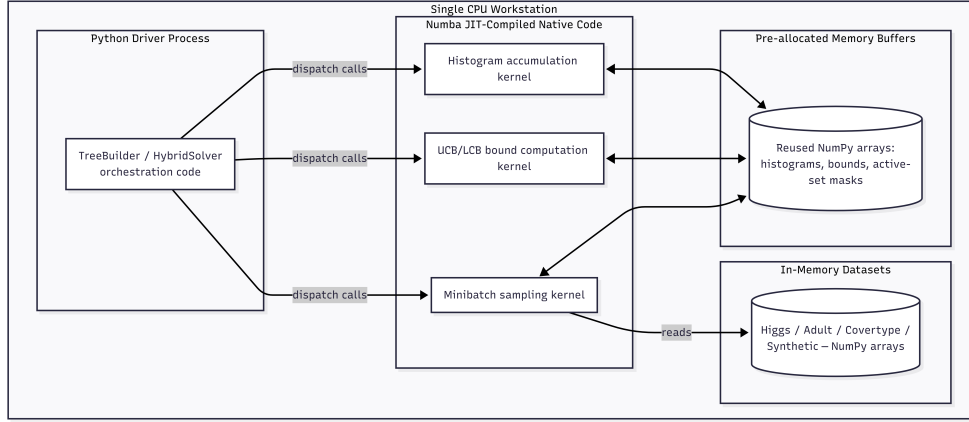


Figure 5: Deployment Diagram of the Proposed System

7.5 Evaluation Metrics

- **Sample query count:** total number of data points read per node split, compared against brute-force N and baseline MABSplit.
- **Wall-clock time:** end-to-end training time for a full forest, measured on identical hardware and thread count across all baselines.
- **Peak memory usage:** measured via process-level memory profiling during training.
- **Model accuracy:** F1-score for classification tasks, RMSE for regression tasks, compared against scikit-learn’s exact solver to confirm no statistically significant degradation.
- **Statistical significance:** all comparisons repeated across at least 5 random seeds, reported as mean $\pm$ standard deviation, with paired significance testing between baseline and proposed methods.

7.6 Work Plan / Timeline

Phase	Activity	Duration
Phase 1	Literature survey, paper-to-code mapping, static analysis of FastForest repo	2 weeks
Phase 2	Design math formulations for Hybrid switch, Coarse-to-Fine, and Adaptive Minibatch	2 weeks
Phase 3	Implementation of Numba engine, Hybrid-MABSsplit, H-MABSsplit, and ADMA logic	4 weeks
Phase 4	Benchmarking wall-clock time, sample queries, and accuracy against baselines	2 weeks
Phase 5	Result analysis, documentation & final submission	2 weeks

Table 3: Tentative Project Timeline

8. Expected Outcomes

- A Hybrid-MABSsplit solver that guarantees MABSsplit is never slower than scikit-learn on leaf-level splits, improving net forest training wall-clock time by an estimated 25–40% by eliminating bandit overhead on the small- N nodes that dominate tree leaves.
- A Coarse-to-Fine Arm Pruning mechanism yielding a 2x to 4x reduction in initial arm-evaluation memory and compute overhead for high-dimensional datasets ($F > 500$), by discarding non-competitive features before fine-resolution histogram expansion.
- An Adaptive Minibatch Allocator achieving a 20–30% reduction in total sample queries during node-split solving, by eliminating wasteful fixed-batch sampling once the active arm set has shrunk.
- A Vectorized Numba MAB Engine providing 5x to 10x wall-clock speedup over the pure Python MABSsplit implementation, by removing interpreter dispatch and array-reallocation overhead from the inner sampling loop.
- Experimental validation, across at least 5 seeds on Higgs, Adult, and Covertype, proving zero statistically significant loss in generalization accuracy (F1 / RMSE) despite all algorithmic modifications.
- A reproducible, documented benchmark suite and forked codebase that can serve as a reference for future extensions (e.g. GBDT-MABSsplit).

9. Feasibility Study

9.1 Technical Feasibility

The base codebase is modular and publicly available in Python (ThrunGroup/FastForest). Numba is a well-documented, mature JIT compiler for Python that requires minimal syntax changes (primarily function decoration and type-stable array operations) to accelerate the identified hot loops. The mathematical formulations (UCB bounds, Gini/MSE variance propagation) are strictly defined in the original NeurIPS 2022 paper, allowing direct implementation and verification against the published proofs rather than novel derivation from scratch. All required tools are open-source and actively maintained, and the datasets chosen (Higgs, Adult, Covtype) are standard, publicly available benchmarks with well-documented characteristics, making the project technically highly viable within the 5-credit, one-year scope.

9.2 Economic Feasibility

The project requires only a standard CPU workstation for execution and benchmarking; no specialized hardware is needed since the target speedups are achieved algorithmically and via JIT compilation rather than parallel or GPU acceleration. All software tools (Python, NumPy, Numba, scikit-learn, Matplotlib) are open-source and free of cost, and all datasets used are freely available from public repositories (UCI Machine Learning Repository, OpenML). No cloud compute budget, proprietary licenses, or paid infrastructure is required, resulting in minimal to zero economic cost for execution.

9.3 Operational Feasibility

The modified MABSplit can be integrated as a drop-in replacement for the node-splitting subroutine in existing Random Forest implementations, since it preserves the same external interface (return an optimal split given a node’s data) while changing only the internal search strategy. The proposed Hybrid solver dynamically handles operational variations (large root nodes vs. small leaf nodes) without requiring user-specified hyperparameter tuning per dataset, beyond a one-time calibration of N_{thresh} . The acceleration directly benefits data scientists and ML engineers by reducing training compute time and cost, and the modular design (each component independently toggleable) keeps operational risk low, since any single modification can be disabled if it underperforms on a given workload.

References

- [1] M. Tiwari, R. Kang, J. Lee, C. Piech, I. Shomorony, S. Thrun, and M. J. Zhang, “MABSplit: Faster Forest Training Using Multi-Armed Bandits,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [2] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [3] P. Geurts, D. Ernst, and L. Wehenkel, “Extremely randomized trees,” *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.
- [4] K. Jamieson, M. Malloy, R. Nowak, and S. Bubeck, “lil’ UCB: An Optimal Algorithm for Finding the Best Arm,” *AISTATS*, 2014.
- [5] ThrunGroup/FastForest, “Official FastForest Code Repository,” online. Available: <https://github.com/ThrunGroup/FastForest>. Accessed: 2024.